RAJALAKSHMI ENGINEERING COLLEGE
Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)
Department of Artificial Intelligence
V Semester B. Tech AIDS

AD23532 – Principles of Data Science

Academic Year: 2026-2027 **Batch:** 2024-2028

Ex 2: Data Discovery and Preparation

Name: Nischal **Date:** 06-10-2026

Register Number: 241801188 **Tools:** Python, Pandas, NumPy, Matplotlib, Seaborn, scikit-learn

Dataset: Telco Customer Churn (IBM)

Aim

To perform data discovery and exploratory data analysis on the Telco Customer Churn dataset: study the distribution of the key features, detect outliers with the IQR method, examine how the features relate to churn, answer the analytical questions about customer behaviour, and prepare the data for predictive analytics by splitting it into training and testing sets and saving the preprocessed dataset.

Task 1: Reuse Ex 1 Steps 1–6 and 10

The preprocessing steps from Ex 1 (import, load, explore, clean, encode, standardise) and the correlation heatmap (Ex 1 Step 10) are repeated below on the same dataset. The cleaned copy `df_clean` (original units) is used for the discovery plots, and `df_encoded` (encoded and scaled) is used for the correlation analysis.

Ex 1 Step 1: Import the required Python libraries

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder, OrdinalEncoder, StandardScaler
from sklearn.model_selection import train_test_split

import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style='whitegrid')
pd.set_option('display.max_columns', None)

print("Libraries imported successfully")
print("pandas:", pd.__version__, "| numpy:", np.__version__, "| seaborn:", sns.__version__)
```

Out [1]:

```
Libraries imported successfully
pandas: 3.0.5 | numpy: 2.5.3 | seaborn: 0.13.2
```

Ex 1 Step 2: Load the Telco Customer Churn dataset into a Pandas DataFrame

In [2]:

```
df = pd.read_csv('Telco-Customer-Churn.csv')
print("Dataset loaded successfully")
```

Out [2]:

```
Dataset loaded successfully
```

2.1 First 10 records

In [3]:

```
df.head(10)
```

Out [3]:

Columns 1–10 of 21

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No
5	9305-CDSKC	Female	0	No	No	8	Yes	Yes	Fiber optic	No
6	1452-KIOVK	Male	0	No	Yes	22	Yes	Yes	Fiber optic	No
7	6713-OKOMC	Female	0	No	No	10	No	No phone service	DSL	Yes
8	7892-POOKP	Female	0	Yes	No	28	Yes	Yes	Fiber optic	No
9	6388-TABGU	Male	0	No	Yes	62	Yes	No	DSL	Yes

Columns 11–17 of 21

	OnlineBackup	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling
0	Yes	No	No	No	No	Month-to-month	Yes
1	No	Yes	No	No	No	One year	No
2	Yes	No	No	No	No	Month-to-month	Yes
3	No	Yes	Yes	No	No	One year	No
4	No	No	No	No	No	Month-to-month	Yes
5	No	Yes	No	Yes	Yes	Month-to-month	Yes
6	Yes	No	No	Yes	No	Month-to-month	Yes
7	No	No	No	No	No	Month-to-month	No
8	No	Yes	Yes	Yes	Yes	Month-to-month	Yes
9	Yes	No	No	No	No	One year	No

Columns 18–21 of 21

	PaymentMethod	MonthlyCharges	TotalCharges	Churn
0	Electronic check	29.85	29.85	No
1	Mailed check	56.95	1889.5	No
2	Mailed check	53.85	108.15	Yes
3	Bank transfer (automatic)	42.30	1840.75	No
4	Electronic check	70.70	151.65	Yes
5	Electronic check	99.65	820.5	Yes
6	Credit card (automatic)	89.10	1949.4	No
7	Mailed check	29.75	301.9	No
8	Electronic check	104.80	3046.05	Yes
9	Bank transfer (automatic)	56.15	3487.95	No

10 rows × 21 columns

2.2 Last 10 records

In [4]:

```
df.tail(10)
```

Out [4]:

Columns 1–9 of 21

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService
7033	9767-FFLEM	Male	0	No	No	38	Yes	No	Fiber optic
7034	0639-TSIQW	Female	0	No	No	67	Yes	Yes	Fiber optic
7035	8456-QDAVC	Male	0	No	No	19	Yes	No	Fiber optic
7036	7750-EYXWZ	Female	0	No	No	12	No	No phone service	DSL
7037	2569-WGERO	Female	0	No	No	72	Yes	No	No
7038	6840-RESVB	Male	0	Yes	Yes	24	Yes	Yes	DSL
7039	2234-XADUH	Female	0	Yes	Yes	72	Yes	Yes	Fiber optic
7040	4801-JZAZL	Female	0	Yes	Yes	11	No	No phone service	DSL
7041	8361-LTMKD	Male	1	Yes	No	4	Yes	Yes	Fiber optic
7042	3186-AJIEK	Male	0	No	No	66	Yes	No	Fiber optic

Columns 10–15 of 21

	OnlineSecurity	OnlineBackup	DeviceProtection	TechSupport	StreamingTV	StreamingMovies
7033	No	No	No	No	No	No
7034	Yes	Yes	Yes	No	Yes	No
7035	No	No	No	No	Yes	No
7036	No	Yes	Yes	Yes	Yes	Yes
7037	No internet service	No internet service	No internet service	No internet service	No internet service	No internet service
7038	Yes	No	Yes	Yes	Yes	Yes
7039	No	Yes	Yes	No	Yes	Yes
7040	Yes	No	No	No	No	No
7041	No	No	No	No	No	No
7042	Yes	No	Yes	Yes	Yes	Yes

Columns 16–21 of 21

	Contract	PaperlessBilling	PaymentMethod	MonthlyCharges	TotalCharges	Churn
7033	Month-to-month	Yes	Credit card (automatic)	69.50	2625.25	No
7034	Month-to-month	Yes	Credit card (automatic)	102.95	6886.25	Yes
7035	Month-to-month	Yes	Bank transfer (automatic)	78.70	1495.1	No
7036	One year	No	Electronic check	60.65	743.3	No
7037	Two year	Yes	Bank transfer (automatic)	21.15	1419.4	No
7038	One year	Yes	Mailed check	84.80	1990.5	No
7039	One year	Yes	Credit card (automatic)	103.20	7362.9	No
7040	Month-to-month	Yes	Electronic check	29.60	346.45	No
7041	Month-to-month	Yes	Mailed check	74.40	306.6	Yes
7042	Two year	Yes	Bank transfer (automatic)	105.65	6844.5	No

10 rows × 21 columns

2.3 Shape of the dataset

In [5]:

```
print("Shape of the dataset:", df.shape)
print("Number of rows :", df.shape[0])
print("Number of columns:", df.shape[1])
```

Out [5]:

```
Shape of the dataset: (7043, 21)
Number of rows : 7043
Number of columns: 21
```

2.4 Column names

In [6]:

```
print("Column names:")
for i, col in enumerate(df.columns, start=1):
    print(f"{i:2d}. {col}")
```

Out [6]:

```
Column names:
 1. customerID
 2. gender
 3. SeniorCitizen
 4. Partner
 5. Dependents
 6. tenure
 7. PhoneService
 8. MultipleLines
 9. InternetService
10. OnlineSecurity
11. OnlineBackup
12. DeviceProtection
13. TechSupport
14. StreamingTV
15. StreamingMovies
16. Contract
17. PaperlessBilling
18. PaymentMethod
19. MonthlyCharges
20. TotalCharges
21. Churn
```

Ex 1 Step 3: Explore the dataset

3.1 Dataset information

In [7]:

```
df.info()
```

Out [7]:

```
<class 'pandas.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   customerID        7043 non-null   str    
 1   gender            7043 non-null   str    
 2   SeniorCitizen     7043 non-null   int64  
 3   Partner           7043 non-null   str    
 4   Dependents        7043 non-null   str    
 5   tenure            7043 non-null   int64  
 6   PhoneService      7043 non-null   str    
 7   MultipleLines     7043 non-null   str    
 8   InternetService   7043 non-null   str    
 9   OnlineSecurity    7043 non-null   str    
 10  OnlineBackup      7043 non-null   str    
 11  DeviceProtection  7043 non-null   str    
 12  TechSupport       7043 non-null   str    
 13  StreamingTV       7043 non-null   str    
 14  StreamingMovies   7043 non-null   str    
 15  Contract          7043 non-null   str    
 16  PaperlessBilling  7043 non-null   str    
 17  PaymentMethod     7043 non-null   str    
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   str    
 20  Churn             7043 non-null   str    
dtypes: float64(1), int64(2), str(18)
memory usage: 1.1 MB
```

3.2 Summary statistics

In [8]:

```
# Summary statistics of numerical columns
df.describe()
```

Out [8]:

	SeniorCitizen	tenure	MonthlyCharges
count	7043.000000	7043.000000	7043.000000
mean	0.162147	32.371149	64.761692
std	0.368612	24.559481	30.090047
min	0.000000	0.000000	18.250000
25%	0.000000	9.000000	35.500000
50%	0.000000	29.000000	70.350000
75%	0.000000	55.000000	89.850000
max	1.000000	72.000000	118.750000

8 rows × 3 columns

In [9]:

```
# Summary statistics of categorical columns
df.describe(exclude='number').T
```

Out [9]:

	count	unique	top	freq
customerID	7043	7043	7590-VHVEG	1
gender	7043	2	Male	3555
Partner	7043	2	No	3641
Dependents	7043	2	No	4933
PhoneService	7043	2	Yes	6361
MultipleLines	7043	3	No	3390
InternetService	7043	3	Fiber optic	3096
OnlineSecurity	7043	3	No	3498
OnlineBackup	7043	3	No	3088
DeviceProtection	7043	3	No	3095
TechSupport	7043	3	No	3473
StreamingTV	7043	3	No	2810
StreamingMovies	7043	3	No	2785
Contract	7043	3	Month-to-month	3875
PaperlessBilling	7043	2	Yes	4171
PaymentMethod	7043	4	Electronic check	2365
TotalCharges	7043	6531	20.2	11
Churn	7043	2	No	5174

18 rows × 4 columns

3.3 Number of missing values

In [10]:

```
missing = df.isnull().sum()
print("Missing values in each column (as read from the file):")
print(missing)
print("\nTotal missing values:", missing.sum())
```

Out [10]:

```
Missing values in each column (as read from the file):
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64

Total missing values: 0
```

Checking for hidden inconsistencies. `TotalCharges` holds amounts but was read as text, which means some rows contain a non-numeric value. These blanks are not counted by `isnull()`.

In [11]:

```
blank_rows = df[df['TotalCharges'].str.strip() == '']
print("Rows with a blank TotalCharges value:", len(blank_rows))
display(blank_rows[['customerID', 'tenure', 'MonthlyCharges', 'TotalCharges']])
```

Out [11]:

Rows with a blank TotalCharges value: 11

	customerID	tenure	MonthlyCharges	TotalCharges
488	4472-LVYGI	0	52.55	
753	3115-CZMZD	0	20.25	
936	5709-LVOEQ	0	80.85	
1082	4367-NUYAO	0	25.75	
1340	1371-DWPAZ	0	56.05	
3331	7644-OMVMY	0	19.85	
3826	3213-VVOLG	0	25.35	
4380	2520-SGTTA	0	20.00	
5218	2923-ARZLG	0	19.70	
6670	4075-WKNIU	0	73.35	
6754	2775-SEFEE	0	61.90	

11 rows × 4 columns

In [12]:

```
# Fix: remove extra spaces from text columns and convert TotalCharges to numeric.
# Blanks become NaN so that they are treated as missing values.
text_cols = df.select_dtypes(exclude='number').columns
df[text_cols] = df[text_cols].apply(lambda s: s.str.strip())
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
missing = df.isnull().sum()
print("Missing values after correcting the data type:")
print(missing[missing > 0])
print("\nTotal missing values:", missing.sum())
```

Out [12]:

```
Missing values after correcting the data type:
TotalCharges    11
dtype: int64

Total missing values: 11
```

> **Observation:** All 11 rows with a blank `TotalCharges` have `tenure = 0`; they are new customers who have not been billed yet. After conversion these blanks become missing values, which are handled in Task 4.

In [13]:

```
# Check the categories of the columns to be encoded for inconsistent labels
for col in ['gender', 'Partner', 'Dependents', 'Contract', 'PaymentMethod', 'InternetService', 'Churn']:
    print(f"{col:16s}: {df[col].unique().tolist()}")
```

Out [13]:

```
gender          : ['Female', 'Male']
Partner         : ['Yes', 'No']
Dependents      : ['No', 'Yes']
Contract        : ['Month-to-month', 'One year', 'Two year']
PaymentMethod   : ['Electronic check', 'Mailed check', 'Bank transfer (automatic)', 'Credit card (automatic)']
InternetService : ['DSL', 'Fiber optic', 'No']
Churn           : ['No', 'Yes']
```

3.4 Number of duplicate records

In [14]:

```
print("Duplicate rows (all columns):", df.duplicated().sum())
print("Duplicate customerID values :", df['customerID'].duplicated().sum())
```

Out [14]:

```
Duplicate rows (all columns): 0
Duplicate customerID values : 0
```

Ex 1 Step 4: Clean the data

4(a) Remove duplicate customer records

In [15]:

```
rows_before = df.shape[0]
df = df.drop_duplicates()                                    # exact duplicate rows
df = df.drop_duplicates(subset='customerID', keep='first')  # same customer listed twice
print("Rows before removing duplicates:", rows_before)
print("Rows after removing duplicates :", df.shape[0])
print("Duplicate records removed      :", rows_before - df.shape[0])
```

Out [15]:

```
Rows before removing duplicates: 7043
Rows after removing duplicates : 7043
Duplicate records removed      : 0
```

4(b) Replace missing values (numerical → median, categorical → mode)

In [16]:

```
num_cols = df.select_dtypes(include='number').columns
cat_cols = df.select_dtypes(exclude='number').columns

# i. Numerical columns -> Median
for col in num_cols:
    n_missing = df[col].isnull().sum()
    if n_missing > 0:
        median_value = df[col].median()
        df[col] = df[col].fillna(median_value)
        print(f"[Numerical] {col}: {n_missing} missing values replaced with median = {median_value:.2f}")

# ii. Categorical columns -> Mode
for col in cat_cols:
    n_missing = df[col].isnull().sum()
    if n_missing > 0:
        mode_value = df[col].mode()[0]
        df[col] = df[col].fillna(mode_value)
        print(f"[Categorical] {col}: {n_missing} missing values replaced with mode = {mode_value}")

print("\nNumerical columns :", list(num_cols))
print("Categorical columns:", len(cat_cols), "columns checked")
```

Out [16]:

```
[Numerical] TotalCharges: 11 missing values replaced with median = 1397.47

Numerical columns : ['SeniorCitizen', 'tenure', 'MonthlyCharges', 'TotalCharges']
Categorical columns: 17 columns checked
```

4(c) Verify that no missing values remain

In [17]:

```
remaining = df.isnull().sum()
print(remaining)
print("\nTotal missing values remaining:", remaining.sum())
if remaining.sum() == 0:
    print("Verified: the dataset has no missing values.")

# Keep a copy of the cleaned data (original units) for the visualisations
df_clean = df.copy()
print("Cleaned dataset shape:", df_clean.shape)
```

Out [17]:

```
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64

Total missing values remaining: 0
Verified: the dataset has no missing values.
Cleaned dataset shape: (7043, 21)
```

Ex 1 Step 5: Encode categorical columns

- **Label Encoding** for binary columns (two categories): `gender`, `Partner`, `Dependents`, `Churn`
- **Ordinal Encoding** for `Contract`, because its categories have a natural order (month-to-month < one year < two year)
- **One-Hot Encoding** for `PaymentMethod` and `InternetService`, because their categories have no order

In [18]:

```
df_encoded = df_clean.copy()

# (a) Label Encoding for binary columns
binary_cols = ['gender', 'Partner', 'Dependents', 'Churn']
le = LabelEncoder()
for col in binary_cols:
    df_encoded[col] = le.fit_transform(df_encoded[col])
    mapping = {cls: i for i, cls in enumerate(le.classes_)}
    print(f"{col:11s}: {mapping}")
```

Out [18]:

```
gender     : {'Female': 0, 'Male': 1}
Partner    : {'No': 0, 'Yes': 1}
Dependents : {'No': 0, 'Yes': 1}
Churn      : {'No': 0, 'Yes': 1}
```

In [19]:

```
# (b) Ordinal Encoding for Contract
contract_order = [['Month-to-month', 'One year', 'Two year']]
oe = OrdinalEncoder(categories=contract_order)
df_encoded['Contract'] = oe.fit_transform(df_encoded[['Contract']]).astype(int)
print("Contract :", {cat: i for i, cat in enumerate(contract_order[0])})
```

Out [19]:

```
Contract : {'Month-to-month': 0, 'One year': 1, 'Two year': 2}
```

In [20]:

```
# (c) One-Hot Encoding for nominal columns with more than two categories
df_encoded = pd.get_dummies(df_encoded, columns=['PaymentMethod', 'InternetService'], dtype=int)

onehot_cols = [c for c in df_encoded.columns if c.startswith(('PaymentMethod_', 'InternetService_'))]
print("New one-hot columns:")
for c in onehot_cols:
    print("  ", c)
print("\nShape after encoding:", df_encoded.shape)
```

Out [20]:

```
New one-hot columns:
   PaymentMethod_Bank transfer (automatic)
   PaymentMethod_Credit card (automatic)
   PaymentMethod_Electronic check
   PaymentMethod_Mailed check
   InternetService_DSL
   InternetService_Fiber optic
   InternetService_No

Shape after encoding: (7043, 26)
```

In [21]:

```
# Encoded columns - first 10 records
encoded_cols = binary_cols + ['Contract'] + onehot_cols
df_encoded[encoded_cols].head(10)
```

Out [21]:

Columns 1–8 of 12

	gender	Partner	Dependents	Churn	Contract	PaymentMethod_Bank transfer (automatic)	PaymentMethod_Credit card (automatic)	PaymentMethod_Electronic check
0	0	1	0	0	0	0	0	1
1	1	0	0	0	1	0	0	0
2	1	0	0	1	0	0	0	0
3	1	0	0	0	1	1	0	0
4	0	0	0	1	0	0	0	1
5	0	0	0	1	0	0	0	1
6	1	0	1	0	0	0	1	0
7	0	0	0	0	0	0	0	0
8	0	1	0	1	0	0	0	1
9	1	0	1	0	1	1	0	0

Columns 9–12 of 12

	PaymentMethod_Mailed check	InternetService_DSL	InternetService_Fiber optic	InternetService_No
0	0	1	0	0
1	1	1	0	0
2	1	1	0	0
3	0	1	0	0
4	0	0	1	0
5	0	0	1	0
6	0	0	1	0
7	1	1	0	0
8	0	0	1	0
9	0	1	0	0

10 rows × 12 columns

Ex 1 Step 6: Standardise the numerical features using StandardScaler

In [22]:

```
scale_cols = ['tenure', 'MonthlyCharges', 'TotalCharges']

print("Before scaling:")
display(df_encoded[scale_cols].describe().round(2))

scaler = StandardScaler()
df_encoded[scale_cols] = scaler.fit_transform(df_encoded[scale_cols])

print("After scaling (mean = 0, standard deviation = 1):")
display(df_encoded[scale_cols].describe().round(2))
```

Out [22]:

Before scaling:

	tenure	MonthlyCharges	TotalCharges
count	7043.00	7043.00	7043.00
mean	32.37	64.76	2281.92
std	24.56	30.09	2265.27
min	0.00	18.25	18.80
25%	9.00	35.50	402.22
50%	29.00	70.35	1397.48
75%	55.00	89.85	3786.60
max	72.00	118.75	8684.80

8 rows × 3 columns

After scaling (mean = 0, standard deviation = 1):

	tenure	MonthlyCharges	TotalCharges
count	7043.00	7043.00	7043.00
mean	-0.00	-0.00	-0.00
std	1.00	1.00	1.00
min	-1.32	-1.55	-1.00
25%	-0.95	-0.97	-0.83
50%	-0.14	0.19	-0.39
75%	0.92	0.83	0.66
max	1.61	1.79	2.83

8 rows × 3 columns

In [23]:

```
# Standardised features - first 10 records
df_encoded[scale_cols].head(10)
```

Out [23]:

	tenure	MonthlyCharges	TotalCharges
0	-1.277445	-1.160323	-0.994242
1	0.066327	-0.259629	-0.173244
2	-1.236724	-0.362660	-0.959674
3	0.514251	-0.746535	-0.194766
4	-1.236724	0.197365	-0.940470
5	-0.992402	1.159546	-0.645186
6	-0.422317	0.808907	-0.146799
7	-0.910961	-1.163647	-0.874137
8	-0.177995	1.330711	0.337349
9	1.206498	-0.286218	0.532439

10 rows × 3 columns

Ex 1 Step 10: Correlation heatmap of all numerical features

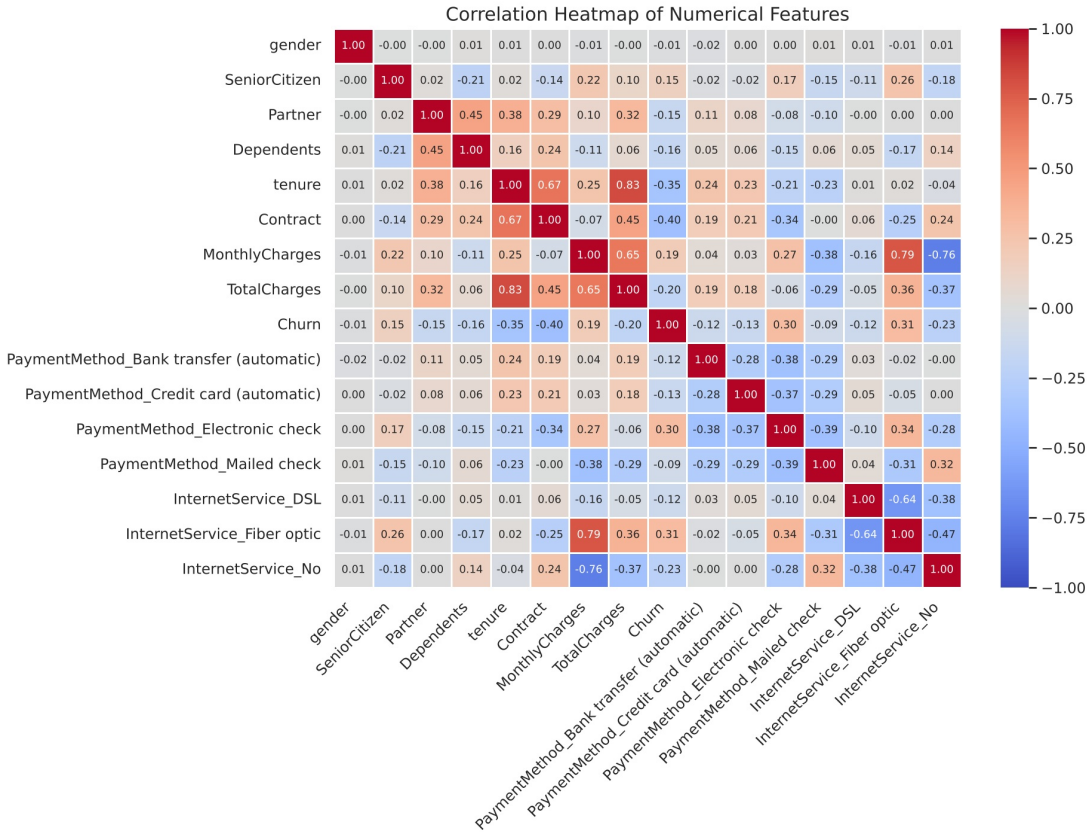
After encoding and scaling, every feature used in this experiment is numerical, so the heatmap covers all numerical columns of the preprocessed dataset.

In [24]:

```
numeric_df = df_encoded.select_dtypes(include='number')
corr = numeric_df.corr()

plt.figure(figsize=(12, 9))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', vmin=-1, vmax=1, center=0,
            linewidths=0.5, annot_kws={'size': 8})
plt.title('Correlation Heatmap of Numerical Features', fontsize=14)
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

Out [24]:



In [25]:

```
# Correlation of each feature with Churn
print(corr['Churn'].drop('Churn').sort_values(ascending=False).round(3))
```

Out [25]:

```
InternetService_Fiber optic               0.308
PaymentMethod_Electronic check            0.302
MonthlyCharges                            0.193
SeniorCitizen                             0.151
gender                                   -0.009
PaymentMethod_Mailed check               -0.092
PaymentMethod_Bank transfer (automatic)  -0.118
InternetService_DSL                      -0.124
PaymentMethod_Credit card (automatic)    -0.134
Partner                                  -0.150
Dependents                               -0.164
TotalCharges                             -0.199
InternetService_No                       -0.228
tenure                                   -0.352
Contract                                 -0.397
Name: Churn, dtype: float64
```

> **Observation:** Churn is most negatively correlated with Contract (-0.40) and tenure (-0.35), and most positively with Fiber optic internet (0.31) and Electronic check payment (0.30). TotalCharges is strongly correlated with tenure (0.83), since it accumulates over time.

Task 2: Distribution of Churn, Gender, Contract Type and Payment Method

In [26]:

```
churn_palette = {'No': '#4C72B0', 'Yes': '#DD8452'}
dist_cols = ['Churn', 'gender', 'Contract', 'PaymentMethod']

fig, axes = plt.subplots(2, 2, figsize=(12, 7.5))
for ax, col in zip(axes.ravel(), dist_cols):
    order = df_clean[col].value_counts().index
    if col == 'Churn':
        sns.countplot(data=df_clean, x=col, hue=col, order=order, palette=churn_palette, legend=False, ax=ax)
    else:
        sns.countplot(data=df_clean, x=col, order=order, color='#4C72B0', ax=ax)
    for container in ax.containers:
        ax.bar_label(container, fontsize=9)
    ax.set_title(f'Distribution of {col}')
    ax.set_ylabel('Number of Customers')
    ax.tick_params(axis='x', rotation=12)
plt.tight_layout()
plt.show()

for col in dist_cols:
    counts = df_clean[col].value_counts()
    pct = (counts / len(df_clean) * 100).round(2)
    print(pd.DataFrame({'Count': counts, 'Percent (%)': pct}))
    print()
```

Out [26]:




```
       Count  Percent (%)
Churn
No      5174        73.46
Yes     1869        26.54

        Count  Percent (%)
gender
Male     3555        50.48
Female   3488        49.52

               Count  Percent (%)
Contract
Month-to-month   3875        55.02
Two year         1695        24.07
One year         1473        20.91

                           Count  Percent (%)
PaymentMethod
Electronic check            2365        33.58
Mailed check                1612        22.89
Bank transfer (automatic)   1544        21.92
Credit card (automatic)     1522        21.61
```

> **Observation:** Only 26.54% of the customers have churned (1869 of 7043), so the target is moderately imbalanced. Gender is almost evenly split (Male 50.48%, Female 49.52%), so it is unlikely to explain churn. Month-to-month contracts are the most common (55.02%) and Electronic check is the most used payment method (33.58%).

Task 3: Distribution of MonthlyCharges, TotalCharges and Tenure

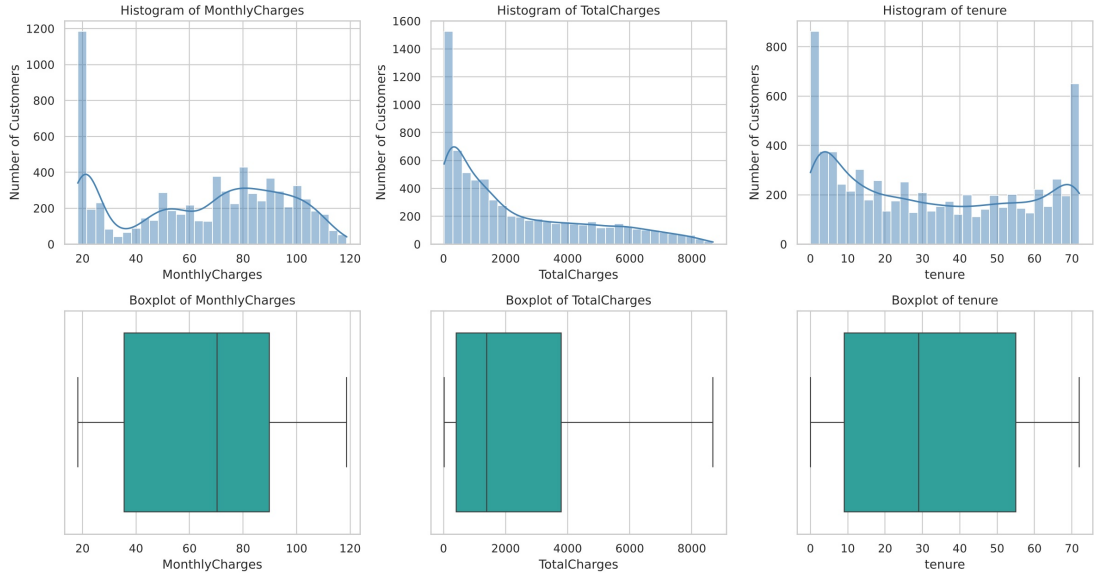
In [27]:

```
num_features = ['MonthlyCharges', 'TotalCharges', 'tenure']

fig, axes = plt.subplots(2, 3, figsize=(15, 8))
for i, col in enumerate(num_features):
    sns.histplot(df_clean[col], bins=30, kde=True, color='steelblue', ax=axes[0, i])
    axes[0, i].set_title(f'Histogram of {col}')
    axes[0, i].set_ylabel('Number of Customers')
    sns.boxplot(x=df_clean[col], color='lightseagreen', ax=axes[1, i])
    axes[1, i].set_title(f'Boxplot of {col}')
plt.tight_layout()
plt.show()

print(df_clean[num_features].agg(['mean', 'median', 'std', 'skew']).round(2))
```

Out [27]:



	MonthlyCharges	TotalCharges	tenure
mean	64.76	2281.92	32.37
median	70.35	1397.48	29.00
std	30.09	2265.27	24.56
skew	-0.22	0.96	0.24

> **Observation:** TotalCharges is strongly right-skewed (skewness 0.96) because many customers are recent and have paid little so far. Tenure has two peaks, one for new customers and one for long-term customers, and MonthlyCharges has a cluster near $20 with a wider spread between $70 and $100. The boxplots show no points beyond the whiskers for any of the three features.

Task 4: Detect outliers using the IQR method

A value is treated as an outlier when it is below `Q1 − 1.5 × IQR` or above `Q3 + 1.5 × IQR`. `SeniorCitizen` is a 0/1 flag, so the IQR rule is applied only to the three continuous features.

In [28]:

```
def iqr_outliers(series):
    q1, q3 = series.quantile(0.25), series.quantile(0.75)
    iqr = q3 - q1
    lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    mask = (series < lower) | (series > upper)
    return q1, q3, iqr, lower, upper, int(mask.sum())

rows = []
for col in num_features:
    q1, q3, iqr, lower, upper, count = iqr_outliers(df_clean[col])
    rows.append({'Feature': col, 'Q1': q1, 'Q3': q3, 'IQR': iqr,
                 'Lower fence': lower, 'Upper fence': upper, 'Outliers': count,
                 'Outliers (%)': round(count / len(df_clean) * 100, 2)})

outlier_table = pd.DataFrame(rows).set_index('Feature').round(2)
display(outlier_table)
print("Total outlier values found:", outlier_table['Outliers'].sum())
```

Out [28]:

Feature	Q1	Q3	IQR	Lower fence	Upper fence	Outliers	Outliers (%)
MonthlyCharges	35.50	89.85	54.35	-46.02	171.38	0	0.0
TotalCharges	402.22	3786.60	3384.38	-4674.34	8863.16	0	0.0
tenure	9.00	55.00	46.00	-60.00	124.00	0	0.0

3 rows × 7 columns

```
Total outlier values found: 0
```

> **Observation:** No outliers were found in tenure, MonthlyCharges or TotalCharges: every value lies inside the IQR fences, so no rows were removed or capped. The right skew of TotalCharges is the natural shape of the data, not an outlier problem.

Task 5: Relationship between the features and Churn

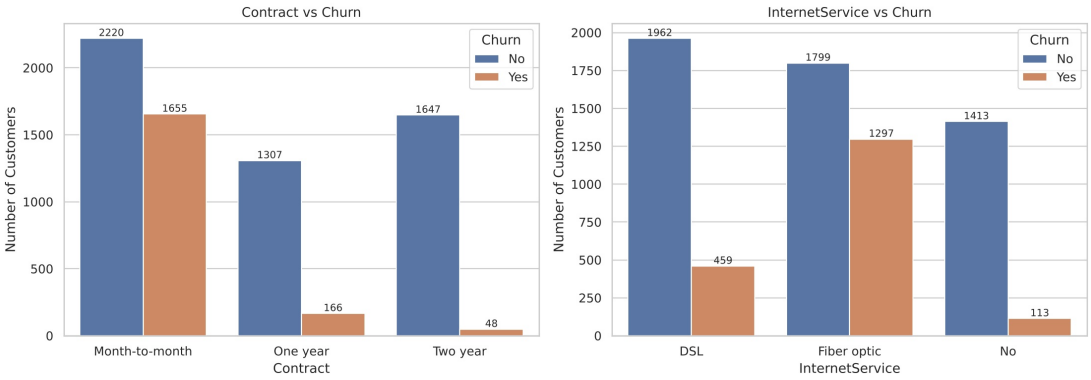
5.1 Contract Type and Internet Service vs Churn (count plots)

In [29]:

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
plot_info = [('Contract', ['Month-to-month', 'One year', 'Two year']),
             ('InternetService', ['DSL', 'Fiber optic', 'No'])]
for ax, (col, order) in zip(axes, plot_info):
    sns.countplot(data=df_clean, x=col, hue='Churn', palette=churn_palette,
                  order=order, hue_order=['No', 'Yes'], ax=ax)
    for container in ax.containers:
        ax.bar_label(container, fontsize=9)
    ax.set_title(f'{col} vs Churn')
    ax.set_ylabel('Number of Customers')
plt.tight_layout()
plt.show()

for col, _ in plot_info:
    rate = pd.crosstab(df_clean[col], df_clean['Churn'], normalize='index') * 100
    print(f"Churn rate (%) by {col}:")
    print(rate.round(2))
    print()
```

Out [29]:




```
Churn rate (%) by Contract:
Churn              No    Yes
Contract
Month-to-month  57.29  42.71
One year        88.73  11.27
Two year        97.17   2.83

Churn rate (%) by InternetService:
Churn                No    Yes
InternetService
DSL              81.04  18.96
Fiber optic      58.11  41.89
No               92.60   7.40
```

5.2 Monthly Charges vs Churn (box plot and histogram)

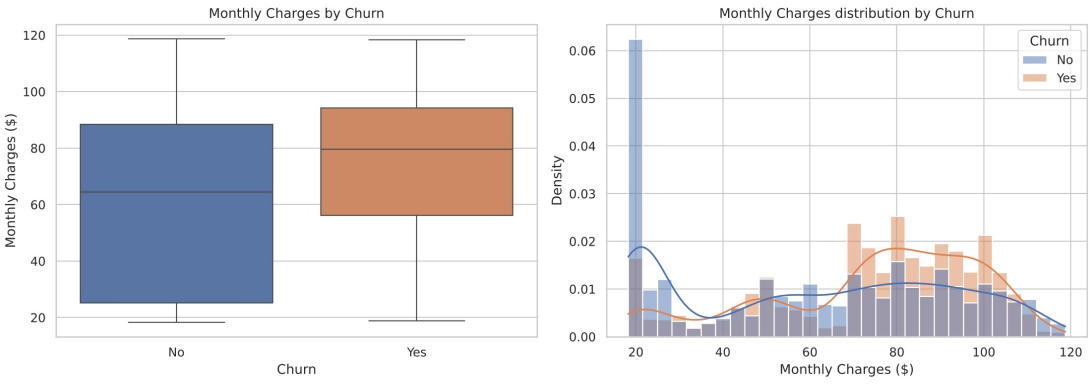
In [30]:

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
sns.boxplot(data=df_clean, x='Churn', y='MonthlyCharges', hue='Churn',
            palette=churn_palette, order=['No', 'Yes'], legend=False, ax=axes[0])
axes[0].set_title('Monthly Charges by Churn')
axes[0].set_ylabel('Monthly Charges ($)')

sns.histplot(data=df_clean, x='MonthlyCharges', hue='Churn', palette=churn_palette,
             bins=30, kde=True, stat='density', common_norm=False, ax=axes[1])
axes[1].set_title('Monthly Charges distribution by Churn')
axes[1].set_xlabel('Monthly Charges ($)')
plt.tight_layout()
plt.show()

print(df_clean.groupby('Churn')['MonthlyCharges'].agg(['mean', 'median', 'std']).round(2))
```

Out [30]:




```
       mean  median    std
```

```
Churn
No     61.27   64.43  31.09
Yes    74.44   79.65  24.67
```

5.3 Tenure vs Churn (box plot and histogram)

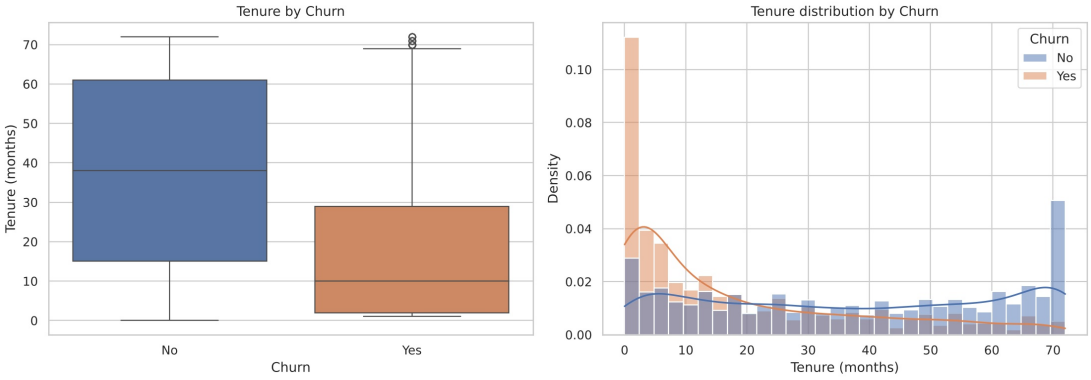
In [31]:

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
sns.boxplot(data=df_clean, x='Churn', y='tenure', hue='Churn',
            palette=churn_palette, order=['No', 'Yes'], legend=False, ax=axes[0])
axes[0].set_title('Tenure by Churn')
axes[0].set_ylabel('Tenure (months)')

sns.histplot(data=df_clean, x='tenure', hue='Churn', palette=churn_palette,
             bins=30, kde=True, stat='density', common_norm=False, ax=axes[1])
axes[1].set_title('Tenure distribution by Churn')
axes[1].set_xlabel('Tenure (months)')
plt.tight_layout()
plt.show()

print(df_clean.groupby('Churn')['tenure'].agg(['mean', 'median', 'std']).round(2))
```

Out [31]:




```
        mean  median    std
Churn
No     37.57    38.0  24.11
Yes    17.98    10.0  19.53
```

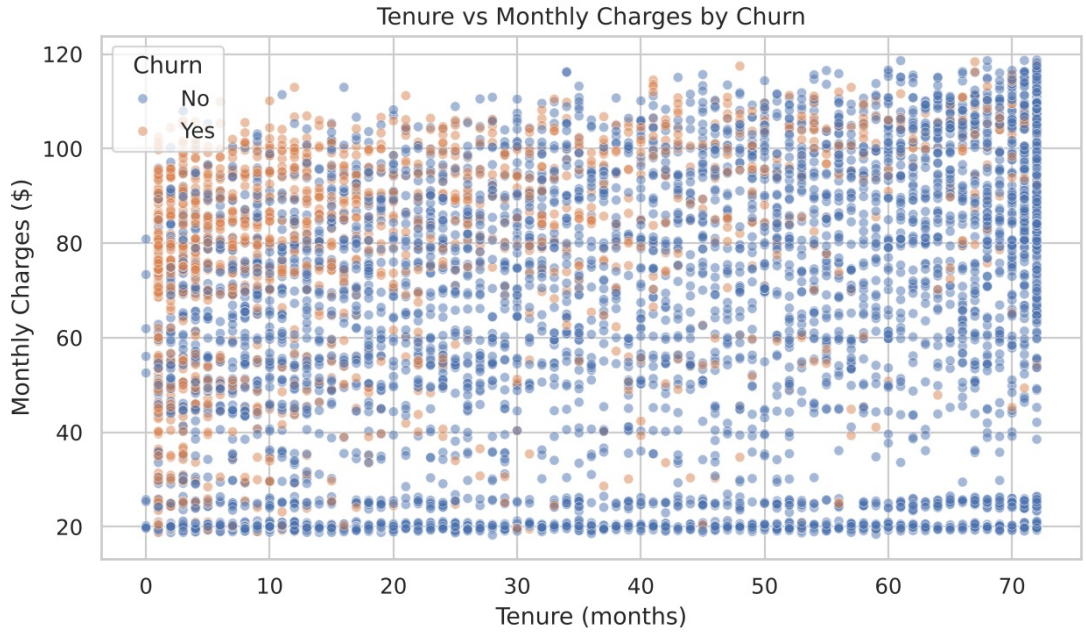
5.4 Tenure vs Monthly Charges coloured by Churn (scatter plot)

In [32]:

```
plt.figure(figsize=(8, 4.8))
sns.scatterplot(data=df_clean, x='tenure', y='MonthlyCharges', hue='Churn',
                palette=churn_palette, hue_order=['No', 'Yes'], alpha=0.5, s=25)
plt.title('Tenure vs Monthly Charges by Churn')
plt.xlabel('Tenure (months)')
plt.ylabel('Monthly Charges ($)')
plt.tight_layout()
plt.show()

churned = df_clean[df_clean['Churn'] == 'Yes']
print("Churners with tenure <= 12 months   : %.2f%%" % ((churned['tenure'] <= 12).mean() * 100))
print("Churners paying more than $70/month : %.2f%%" % ((churned['MonthlyCharges'] > 70).mean() * 100))
```

Out [32]:




```
Churners with tenure <= 12 months   : 55.48%
```

```
Churners paying more than $70/month : 67.79%
```

> **Observation:** Churn is concentrated among customers with short tenure and higher monthly charges: 55.48% of churners have a tenure of 12 months or less and 67.79% pay more than $70 per month. Customers who stay appear across the whole range, including long tenures with low charges.

Task 6: Analytical questions

To compare every attribute with Churn, the remaining categorical columns (for example PhoneService, OnlineSecurity, TechSupport) are also one-hot encoded and `customerID` is dropped, because it is only an identifier. In the add-on service columns the labels 'No internet service' and 'No phone service' are merged into 'No', because that information is already held in InternetService and PhoneService. This model-ready dataset, `df_final`, is the one split in Task 7 and saved in Task 8.

In [33]:

```
df_final = df_encoded.drop(columns='customerID')

# 'No internet service' / 'No phone service' mean the same as 'No' for these add-on columns
# (that information is already held in InternetService and PhoneService)
addon_cols = ['MultipleLines', 'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
              'TechSupport', 'StreamingTV', 'StreamingMovies']
df_final[addon_cols] = df_final[addon_cols].replace({'No internet service': 'No', 'No phone service': 'No'})

remaining_cat = df_final.select_dtypes(exclude='number').columns.tolist()
print("Remaining categorical columns:", remaining_cat)

df_final = pd.get_dummies(df_final, columns=remaining_cat, drop_first=True, dtype=int)
print("\nShape of the model-ready dataset:", df_final.shape)
print("Non-numeric columns left        :", df_final.select_dtypes(exclude='number').shape[1])
print("Missing values                  :", int(df_final.isnull().sum().sum()))
```

Out [33]:

```
Remaining categorical columns: ['PhoneService', 'MultipleLines', 'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport',
'StreamingTV', 'StreamingMovies', 'PaperlessBilling']

Shape of the model-ready dataset: (7043, 25)
Non-numeric columns left        : 0
Missing values                  : 0
```

Q1. Which contract type has the highest customer churn?

In [34]:

```
contract_churn = (pd.crosstab(df_clean['Contract'], df_clean['Churn'], normalize='index')['Yes'] * 100).round(2)
print("Churn rate (%) by contract type:")
print(contract_churn.sort_values(ascending=False))
print("\nHighest churn:", contract_churn.idxmax(), f"({contract_churn.max():.2f}%)")
```

Out [34]:

```
Churn rate (%) by contract type:
Contract
Month-to-month    42.71
One year          11.27
Two year           2.83
Name: Yes, dtype: float64

Highest churn: Month-to-month (42.71%)
```

> **Answer:** The **Month-to-month** contract has the highest churn at 42.71%, compared with 11.27% for one-year and 2.83% for two-year contracts.

Q2. Which payment method is most frequently used by customers who churn?

In [35]:

```
churned = df_clean[df_clean['Churn'] == 'Yes']
pay_counts = churned['PaymentMethod'].value_counts()
pay_rate = (pd.crosstab(df_clean['PaymentMethod'], df_clean['Churn'], normalize='index')['Yes'] * 100)
pay_summary = pd.DataFrame({'Churned customers': pay_counts,
                            'Share of all churners (%)': (pay_counts / len(churned) * 100).round(2),
                            'Churn rate within method (%)': pay_rate.round(2)})
display(pay_summary)
print("Most used by churners:", pay_counts.idxmax())
```

Out [35]:

PaymentMethod	Churned customers	Share of all churners (%)	Churn rate within method (%)
Bank transfer (automatic)	258	13.80	16.71
Credit card (automatic)	232	12.41	15.24
Electronic check	1071	57.30	45.29
Mailed check	308	16.48	19.11

4 rows × 3 columns

```
Most used by churners: Electronic check
```

> **Answer: Electronic check** is the most frequently used payment method among churners (1071 customers, 57.30% of all churners), and it also has the highest churn rate within a method (45.29%).

Q3. Do customers with shorter tenure tend to churn more often?

In [36]:

```
print("Tenure (months) by churn status:")
print(df_clean.groupby('Churn')['tenure'].agg(['mean', 'median']).round(2))

tenure_group = pd.cut(df_clean['tenure'], bins=[-1, 12, 24, 48, 72],
                      labels=['0-12', '13-24', '25-48', '49-72'])
tenure_churn = (pd.crosstab(tenure_group, df_clean['Churn'], normalize='index')['Yes'] * 100).round(2)
print("\nChurn rate (%) by tenure group (months):")
print(tenure_churn)
```

Out [36]:

```
Tenure (months) by churn status:
       mean  median
Churn
No     37.57    38.0
Yes    17.98    10.0

Churn rate (%) by tenure group (months):
tenure
0-12     47.44
13-24    28.71
25-48    20.39
49-72     9.51
Name: Yes, dtype: float64
```

> **Answer:** Yes. Churners have a median tenure of 10 months against 38 months for retained customers, and the churn rate falls steadily as tenure grows, from 47.44% in the first 12 months to 9.51% for customers with more than 48 months.

Q4. Which customer group has the highest monthly charges?

In [37]:

```
group_cols = ['InternetService', 'Contract', 'PaymentMethod', 'Churn',
              'SeniorCitizen', 'Partner', 'Dependents', 'gender']
records = []
for col in group_cols:
    for grp, avg in df_clean.groupby(col)['MonthlyCharges'].mean().items():
        records.append((col, str(grp), round(avg, 2)))

group_means = (pd.DataFrame(records, columns=['Feature', 'Group', 'Avg MonthlyCharges'])
               .sort_values('Avg MonthlyCharges', ascending=False).reset_index(drop=True))
display(group_means.head(8))
```

Out [37]:

	Feature	Group	Avg MonthlyCharges
0	InternetService	Fiber optic	91.50
1	SeniorCitizen	1	79.82
2	PaymentMethod	Electronic check	76.26
3	Churn	Yes	74.44
4	Partner	Yes	67.78
5	PaymentMethod	Bank transfer (automatic)	67.19
6	Dependents	No	67.00
7	PaymentMethod	Credit card (automatic)	66.51

8 rows × 3 columns

> **Answer:** Among the customer groups examined, the highest average monthly charge belongs to customers with **InternetService = Fiber optic** (about $91.50 per month). Customers who churn also pay more on average ($74.44) than those who stay ($61.27).

Q5. Which Internet Service category experiences the highest churn?

In [38]:

```
internet_churn = (pd.crosstab(df_clean['InternetService'], df_clean['Churn'], normalize='index')['Yes'] * 100).round(2)
print("Churn rate (%) by Internet Service:")
print(internet_churn.sort_values(ascending=False))
```

Out [38]:

```
Churn rate (%) by Internet Service:
InternetService
Fiber optic    41.89
DSL            18.96
No              7.40
Name: Yes, dtype: float64
```

> **Answer: Fiber optic** customers churn the most (41.89%), well above DSL (18.96%) and customers with no internet service (7.40%).

Q6. Is there any relationship between monthly charges and churn?

In [39]:

```
churn_flag = (df_clean['Churn'] == 'Yes').astype(int)
print("Correlation between MonthlyCharges and Churn: %.3f" % df_clean['MonthlyCharges'].corr(churn_flag))

charge_band = pd.qcut(df_clean['MonthlyCharges'], 4,
                      labels=['Q1 (lowest)', 'Q2', 'Q3', 'Q4 (highest)'])
band_churn = (pd.crosstab(charge_band, df_clean['Churn'], normalize='index')['Yes'] * 100).round(2)
print("\nChurn rate (%) by monthly charge quartile:")
print(band_churn)
```

Out [39]:

```
Correlation between MonthlyCharges and Churn: 0.193

Churn rate (%) by monthly charge quartile:
MonthlyCharges
Q1 (lowest)     11.24
Q2              24.58
Q3              37.51
Q4 (highest)    32.88
Name: Yes, dtype: float64
```

> **Answer:** Yes, there is a positive but weak relationship (correlation 0.193). Customers in the highest charge quartile churn at 32.88%, compared with 11.24% in the lowest quartile, so higher bills go with higher churn.

Q7. Which features appear to be the strongest indicators of customer churn?

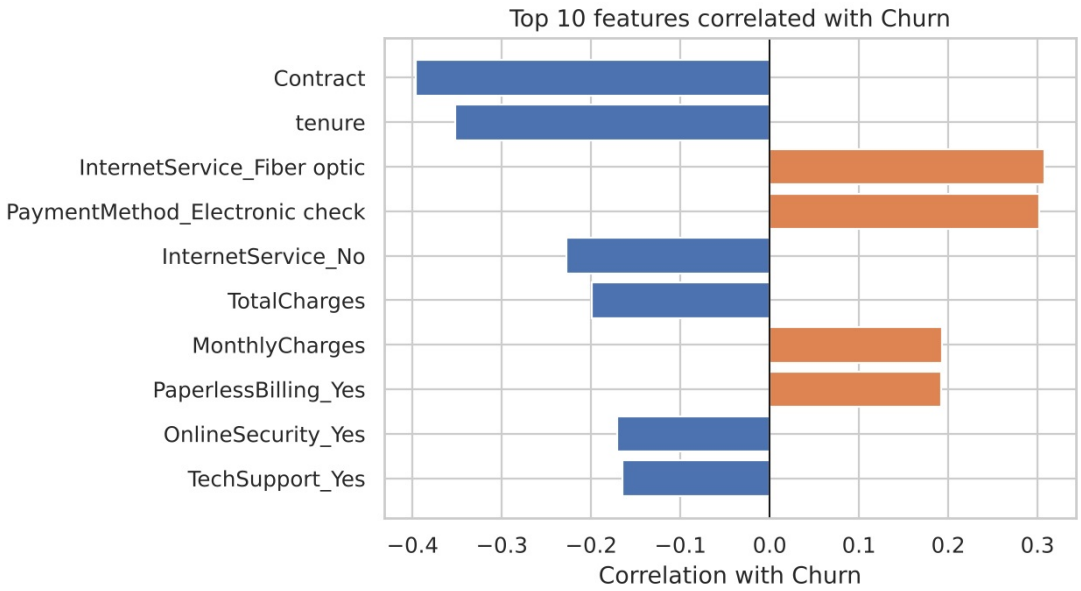
In [40]:

```
churn_corr = df_final.corr()['Churn'].drop('Churn')
top_features = churn_corr.reindex(churn_corr.abs().sort_values(ascending=False).index).head(10)
print("Top 10 features by absolute correlation with Churn:")
print(top_features.round(3))

plt.figure(figsize=(8, 4.5))
colors = ['#DD8452' if v > 0 else '#4C72B0' for v in top_features.values[::-1]]
plt.barh(top_features.index[::-1], top_features.values[::-1], color=colors)
plt.axvline(0, color='black', linewidth=0.8)
plt.title('Top 10 features correlated with Churn')
plt.xlabel('Correlation with Churn')
plt.tight_layout()
plt.show()
```

Out [40]:

```
Top 10 features by absolute correlation with Churn:
Contract                         -0.397
tenure                           -0.352
InternetService_Fiber optic       0.308
PaymentMethod_Electronic check    0.302
InternetService_No               -0.228
TotalCharges                     -0.199
MonthlyCharges                    0.193
PaperlessBilling_Yes              0.192
OnlineSecurity_Yes               -0.171
TechSupport_Yes                  -0.165
Name: Churn, dtype: float64
```



> **Answer:** The strongest indicators are **Contract** (-0.40), **tenure** (-0.35), **InternetService_Fiber optic** (+0.31), **PaymentMethod_Electronic check** (+0.30), **InternetService_No** (-0.23). Short tenure, month-to-month contracts, fibre-optic internet and electronic-check payment go with higher churn, while longer contracts and long tenure go with customers staying. Customers with online security (-0.17) or tech support (-0.16) also churn less. Correlation shows association, not cause.

Task 7: Split the cleaned dataset into training (80%) and testing (20%)

In [41]:

```
X = df_final.drop(columns='Churn')
y = df_final['Churn']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

print("Training set :", X_train.shape, "->", round(len(X_train) / len(X) * 100, 1), "% of the data")
print("Testing set  :", X_test.shape, "->", round(len(X_test) / len(X) * 100, 1), "% of the data")
print("\nChurn rate in the full data  : %.2f%%" % (y.mean() * 100))
print("Churn rate in training set   : %.2f%%" % (y_train.mean() * 100))
print("Churn rate in testing set    : %.2f%%" % (y_test.mean() * 100))
```

Out [41]:

```
Training set : (5634, 24) -> 80.0 % of the data
Testing set  : (1409, 24) -> 20.0 % of the data

Churn rate in the full data  : 26.54%
Churn rate in training set   : 26.54%
Churn rate in testing set    : 26.54%
```

> **Note:** The split is stratified on Churn with `random_state=42`, so both parts keep the same churn rate and the result is repeatable. Following Ex 1, the scaling was done before the split; in a full modelling pipeline the scaler should be fitted on the training set only, to avoid leaking test-set information.

Task 8: Save the final processed dataset

In [42]:

```
df_final.to_csv('CustomerChurn_Preprocessed.csv', index=False)

saved = pd.read_csv('CustomerChurn_Preprocessed.csv')
print("File saved: CustomerChurn_Preprocessed.csv")
print("Shape of saved file :", saved.shape)
print("Missing values      :", int(saved.isnull().sum().sum()))
print("Identical to df_final:", saved.shape == df_final.shape and bool((saved.columns == df_final.columns).all()))
saved.head(5)
```

Out [42]:

```
File saved: CustomerChurn_Preprocessed.csv
Shape of saved file : (7043, 25)
Missing values      : 0
Identical to df_final: True
```

Columns 1–9 of 25

	gender	SeniorCitizen	Partner	Dependents	tenure	Contract	MonthlyCharges	TotalCharges	Churn
0	0	0	1	0	-1.277445	0	-1.160323	-0.994242	0
1	1	0	0	0	0.066327	1	-0.259629	-0.173244	0
2	1	0	0	0	-1.236724	0	-0.362660	-0.959674	1
3	1	0	0	0	0.514251	1	-0.746535	-0.194766	0
4	0	0	0	0	-1.236724	0	0.197365	-0.940470	1

Columns 10–14 of 25

	PaymentMethod_Bank transfer (automatic)	PaymentMethod_Credit card (automatic)	PaymentMethod_Electronic check	PaymentMethod_Mailed check	InternetService_DSL
0	0	0	1	0	1
1	0	0	0	1	1
2	0	0	0	1	1
3	1	0	0	0	1
4	0	0	1	0	0

Columns 15–20 of 25

	InternetService_Fiber optic	InternetService_No	PhoneService_Yes	MultipleLines_Yes	OnlineSecurity_Yes	OnlineBackup_Yes
0	0	0	0	0	0	1
1	0	0	1	0	1	0
2	0	0	1	0	1	1
3	0	0	0	0	1	0
4	1	0	1	0	0	0

Columns 21–25 of 25

	DeviceProtection_Yes	TechSupport_Yes	StreamingTV_Yes	StreamingMovies_Yes	PaperlessBilling_Yes
0	0	0	0	0	1
1	1	0	0	0	0
2	0	0	0	0	1
3	1	1	0	0	0
4	0	0	0	0	1

5 rows × 25 columns

> **Observation:** The saved file has 7043 rows and 25 columns (24 features plus the Churn target), contains no missing values and is fully numeric, so it can be used directly to train a churn prediction model.

Result

Thus data discovery and exploratory analysis were carried out on the Telco Customer Churn dataset (7,043 records, 21 attributes). The distributions of churn, gender, contract type and payment method were studied, the continuous features were checked for outliers using the IQR method, and the relationships between the main features and churn were visualised. Month-to-month contracts, short tenure, fibre-optic internet, electronic-check payment and higher monthly charges were found to go with higher churn. The cleaned and encoded dataset was split into 80% training and 20% testing sets and saved as CustomerChurn_Preprocessed.csv, ready for building a churn prediction model.